WORKLET 2

CYK Structural Parser & Validation Engine

Technical Specification & Project Documentation

1. Executive Summary & Overview

Worklet 2 is a high-performance C++ grammar parser and structural validation engine. It accepts a stream of tokens produced by Worklet 1, validates them against Chomsky Normal Form (CNF) grammar rules using the Cocke-Younger-Kasami (CYK) dynamic programming algorithm, performs AST tree reconstruction, and enforces semantic security constraints such as delimiter matching and nesting depth limits.

2. Team Responsibilities & Repository Structure

The project workload is distributed across four team members mapping directly to the directory layout:

Member 1: Grammar Core & Configuration

- **Owned Files:** `config/cfg_rules.json, include/worklet2/grammar/, src/grammar/`
- **Responsibilities:** Parses grammar configuration via nlohmann/json.hpp, validates Chomsky Normal Form (CNF) rules, and builds O(1) fast lookup indices using custom hashed RuleKey structs.

Member 2: CYK Dynamic Programming Engine

- **Owned Files:** `include/worklet2/cyk/, src/cyk/, tests/bench_cyk.cpp`
- **Responsibilities:** Manages the triangular dynamic programming matrix (Cell and RuleMatch backpointers) and executes the $O(n^3 \cdot |G|)$ CYK parsing algorithm.

Member 3: Parse Tree Reconstruction & Semantic Validation

- **Owned Files:** `include/worklet2/tree/, src/tree/`
- **Responsibilities:** Rebuilds Abstract Syntax Trees (AST) from CYK backpointers, verifies semantic nesting depth, and ensures delimiter matching ({} and []).

Member 4: Pipeline Integration & Gateway

- **Owned Files:** `CMakeLists.txt, include/worklet2/common/, include/worklet2/pipeline/, src/main.cpp, tests/test_pipeline.cpp`
- **Responsibilities:** Coordinates the orchestration pipeline (Worklet2Gateway and StructuralValidator), manages shared common types (Token, ValidationResult), and leads system QA/edge-case testing.

3. Input Token Specification (from Worklet 1)

Worklet 2 consumes tokenized streams conforming to Worklet 1's lexical specifications:

Token Type	Category / Match Example
LBRACE / RBRACE	{ / }
LBRACKET / RBRACKET	[/]
COLON / COMMA	: / ,
STRING	Text in double quotes (e.g., "bob")
NUMBER	Integers or decimals (e.g., 25, -3.14)
TRUE / FALSE / NULL	true, false, null

4. Directory Architecture Layout

```
worklet2/
|-- CMakeLists.txt
|-- config/
|   `-- cfg_rules.json          # CNF grammar rules
|-- include/worklet2/
|   |-- common/
|   |   |-- Token.hpp           # Shared token types with Worklet 1
|   |   `-- ValidationResult.hpp  # Session ID, action, parse tree, reject reason
|   |-- grammar/                # Member 1
|   |-- cyk/                    # Member 2
|   |-- tree/                   # Member 3
|   `-- pipeline/               # Member 4
|-- src/                        # Implementation files matching headers
`-- tests/                      # Unit & integration test suites
```

5. Build and Execution Instructions

- **Prerequisites:** C++17 compatible compiler and CMake (version 3.14+).
- **Compilation:**

```
mkdir build && cd build
cmake ..
cmake --build .
```

- **Execution:** Run the binary executable `./worklet2_demo` to verify pipeline orchestration.